THE OPEN SOURCE AUDIT

A Capstone Project | Open Source Software NGMC Course

Chosen Software: Mozilla Firefox

Student Name: Devang Atiyolil

Roll Number: 24BCE11332

License: Mozilla Public License 2.0 (MPL 2.0)

Introduction

When I was picking a project for this audit, the choice came down to something I actually use every day versus something that sounded impressive on paper. Firefox won, not because it is the most technically complex option on the list, but because it represents something genuinely unusual in the software world: a nonprofit organisation going up against billion-dollar corporations and, for the most part, holding its ground.

This report covers why Firefox exists, what its license actually means, where it lives on a Linux system, the ecosystem it is part of, and an honest comparison with its proprietary competition. Alongside the written analysis, I have written five shell scripts that demonstrate practical Linux skills tied to the themes of each section.

The goal of this audit is not to fanboy over Firefox. It is to actually understand why someone built it, who they built it for, and what it means that it is open source. Those questions turn out to be more interesting than I expected going in.

Part A -- Origin and Philosophy (Units 1 & 2)

A1. The Problem Firefox Was Created to Solve

To understand why Firefox exists, you have to remember what the web looked like in the early 2000s. Internet Explorer was not just the most popular browser, it was essentially the only browser. Microsoft had won the first browser war by bundling IE with Windows and pushing Netscape out of the market. By 2002, IE held somewhere around 95 percent of global market share. That kind of dominance would be impressive if the product were good. It was not.

IE was slow, insecure, and deeply hostile to web standards. Microsoft had won the war and then stopped trying. Security vulnerabilities piled up. Pages looked different depending on which browser rendered them because IE kept inventing its own rules rather than following the open standards that the W3C was trying to establish. Developers spent enormous time writing workaround code just to make things render correctly.

Netscape, the company that had once owned the browser market, was dying. In a last attempt to remain relevant, it released its browser's source code to the public in 1998 and formed the Mozilla project. The idea was that the open-source community could take the code, improve it, and build something better than any single company could. The early result, the Mozilla Suite, was bloated and slow. It tried to be everything at once.

A group of volunteers inside the Mozilla project decided the right move was to strip everything down and build only a browser. Fast, standards-compliant, and built with the user in mind rather than the interests of an operating system vendor. That project became Phoenix, then Firebird, then Firefox. Version 1.0 launched in November 2004 and gained ten million downloads in its first ten days.

The problem Firefox was solving was not purely technical. Other browsers existed. The problem was that the most-used browser was controlled by one company with no real incentive to improve it, and that company was using its market position to lock the web into its own proprietary interpretation of how things should work. Firefox was the answer to the question: what happens when the open-source community decides a piece of critical infrastructure is too important to leave in one company's hands?

Mozilla Corporation, the subsidiary that manages Firefox's development, is owned by the Mozilla Foundation, a nonprofit. That structure is deliberate. It means decisions about Firefox are not made primarily to maximise shareholder returns. They are made, at least in principle, to serve the open web and the people who use it. That is an unusual thing in software and it is the main reason Firefox is worth studying carefully.

A2. The License: What MPL 2.0 Actually Says

The Four Freedoms

Richard Stallman's definition of free software rests on four freedoms. Freedom zero is the freedom to run the program for any purpose. Freedom one is the freedom to study how it works and change it. Freedom two is the freedom to redistribute copies. Freedom three is the freedom to distribute your modified versions. Firefox's license, the Mozilla Public License 2.0, grants all four of these freedoms. You can download the source code, read every line of it, modify it, and distribute your version. There are conditions attached, but the core freedoms are present.

Can a Company Take Firefox, Modify It, and Sell It Without Sharing Changes?

This is where MPL 2.0 gets interesting and where it differs from GPL. MPL 2.0 is a file-level copyleft license. The copyleft obligation applies only to the specific files you modify. If you take a Firefox source file, change it, and distribute your product, you must share those changed files under MPL 2.0. But if you write entirely new files and combine them with Firefox, those new files are yours to keep proprietary.

So yes, a company can build a proprietary product on top of Firefox. Several have done exactly that. The condition is that any modifications to the existing MPL-licensed files must be shared. This is a deliberate middle ground. Mozilla wanted Firefox to be usable by companies without requiring them to open-source everything they built, while still ensuring that improvements to the core browser stayed open.

GPL vs MIT: The Key Difference

GPL is a strong copyleft license. If you modify GPL code and distribute the result, the entire combined work must also be licensed under GPL. This is sometimes called viral licensing because the requirement spreads to all the code it touches. MIT is permissive. It essentially says: do whatever you want with this, just keep the copyright notice. You can take MIT code, close the source entirely, and sell it without any obligation to give back.

If I were choosing a license for something I built, my choice would depend on what I wanted to happen with it. If I were building a library that I wanted developers to actually adopt, I would pick MIT. Adoption matters more than enforcing openness when the goal is to get the tool used. If I were building something I considered critical infrastructure, something I did not want a corporation to

quietly absorb and close, I would pick GPL. The license is a statement about values, not just terms and conditions.

Free as in Beer vs Free as in Freedom

This distinction comes from Stallman and it is worth understanding properly. Free as in beer means it costs nothing to acquire. Free as in freedom means you have the rights to use, study, modify, and share it. Firefox is both, but the second kind of freedom is the one that actually matters here.

A concrete example: you can download a pre-built Firefox binary for free. That is free beer. But you can also go to the Mozilla source repository, read every line of code that runs when you open a tab, change the part you do not like, compile it, and use your modified version. That is freedom. Google Chrome is free beer. It costs nothing. But you cannot read all of its source code, you cannot modify how it handles your data, and you cannot redistribute a changed version. Chromium, the open-source base Chrome is built on, is open. The actual Chrome product has proprietary layers on top that you do not control and cannot audit.

A3. The Ethics of Open Source

Should All Software Be Open Source?

The argument for it is strong. Software is not like a physical product. When you share code, you do not lose it. The marginal cost of giving someone a copy is zero. If all software were open, every developer could learn from every other developer's work, security vulnerabilities would be found faster because more eyes could look at the code, and no single company could lock users into a product they cannot leave or inspect.

The argument against it is also real. Some software requires enormous investment to build. A company that spends years and millions of dollars developing something genuinely novel cannot survive if competitors can simply copy the result the day it ships. Closed source protects that investment. There is also the argument that certain types of security software benefit in specific narrow ways from obscurity.

My position is that software running critical public infrastructure should be open. Browsers, operating systems, databases, compilers. The web browser is where people access information, do their banking, and communicate. That seems too important to be controlled by a company whose business model is tracking behavior for advertising revenue.

Is It Ethical to Profit from Community Labor?

Red Hat built a billion-dollar business on Linux. Google runs its entire infrastructure on Linux and open-source tools. Meta does the same. None of them invented Linux. They took something built by thousands of volunteers and turned it into profit.

Whether that is ethical depends on what they gave back. Red Hat contributes substantially to the Linux kernel. Google employs engineers who work on open-source projects full time and releases enormous amounts of its own infrastructure as open source. Meta created React, which is now one of the most widely used front-end frameworks in the world. These are real contributions that go back to the community.

The cases that feel troubling are companies that take open-source work, build on top of it, make money, and give nothing back. No code, no funding, not even acknowledgment. That is legal under

permissive licenses. Whether it is ethical is a different question. If your product would not exist without community labor, some portion of what you earn should flow back to sustaining that community. That is not a legal position, just a reasonable one.

Standing on the Shoulders of Giants

In software, almost no one builds from nothing. Every application runs on an operating system someone else wrote. Every web app runs on a browser engine that teams of engineers spent years building. Every Python script runs on an interpreter maintained by a community. The metaphor is accurate. The question is whether open source enables or undermines original innovation.

It enables it. The reason someone can build a functional web application in a weekend is because they do not have to write their own HTTP server, their own TLS implementation, their own database, or their own rendering engine. All of that exists, is open, and can be used freely. Open source raises the floor for everyone. The ceiling for truly original work is actually higher because you start from further up.

Part B -- Linux Footprint (Unit 2)

Installation Method

Firefox is available through the package manager on virtually every major Linux distribution. On Debian and Ubuntu based systems it installs via apt. On Fedora and RHEL based systems it installs via dnf or yum. On Arch it comes through pacman.

- Debian/Ubuntu: sudo apt install firefox or sudo apt install firefox-esr
- Fedora/RHEL: sudo dnf install firefox
- Arch Linux: sudo pacman -S firefox

Mozilla also distributes official tarballs from their website for users who want the latest release independent of their distro's package cycle. These extract and run directly without a system-level install, which is useful on enterprise systems where the distro's packaged version lags behind Mozilla's release schedule.

Key Directories

Once installed, Firefox occupies several locations across the Linux filesystem hierarchy.

- /usr/bin/firefox -- the main executable called when you launch the browser
- /usr/lib/firefox/ -- the core application files, shared libraries, and the browser engine itself
- /usr/share/applications/firefox.desktop -- the .desktop file that tells the system how to display Firefox in application menus and launchers
- /etc/firefox/ -- system-wide configuration and policy files, used in enterprise deployments to enforce settings across all users on the machine

- ~/.mozilla/firefox/ -- per-user profile directory containing bookmarks, history, saved passwords, extensions, and session data
- ~/.cache/mozilla/firefox/ -- browser cache, can grow large over time, safe to delete to reclaim space

The separation between /usr/lib (system files, identical for all users) and ~/.mozilla (user-specific data) follows standard Linux filesystem conventions and matters both for security and for supporting multiple users on the same machine.

User and Group

Firefox runs as the currently logged-in user, not as root and not as a dedicated service account. This matters for security in two ways. First, if the browser is compromised, the attacker has the same permissions as your user account and not root permissions. That is bad, but significantly less bad than a root-level compromise. Second, each user's profile data under ~/.mozilla is readable only by that user, which provides meaningful privacy on shared systems.

The system files under /usr/lib/firefox are owned by root and are not writable by regular users. You can read and execute them but not modify them directly. This is the correct behavior for system-installed applications and prevents a compromised user session from modifying the browser binary itself.

Service Management

Firefox is a desktop application, not a system service. It does not run as a daemon and is not managed by systemd in the way a web server or database would be. You start it by clicking the icon or running firefox in a terminal. There is no systemctl start firefox.

In enterprise and kiosk deployments, Firefox can be wrapped in a systemd user service to control startup behavior, but for standard personal use, service management does not apply to Firefox directly.

Update Model

On Linux, Firefox updates come through two channels. The first is the distro package manager. When Mozilla releases a security patch, it gets packaged by distribution maintainers and pushed through the standard update mechanism. Running sudo apt upgrade or sudo dnf update pulls in Firefox updates along with everything else on the system.

The second channel is Mozilla's own built-in updater, which downloads and installs newer versions directly. In enterprise environments this is often disabled in favor of controlled distro updates. For home users both channels work, and Mozilla's built-in updater tends to deliver the latest version faster than waiting for the distro to package it.

Part C -- The FOSS Ecosystem (Units 3 & 4)

Dependencies

Firefox is built on a large stack of open-source components, most maintained internally by Mozilla but connected to the wider ecosystem.

- SpiderMonkey -- Mozilla's own JavaScript engine, developed alongside Firefox and used in some other projects
- WebRender -- a GPU-based rendering engine written in Rust, developed as part of Project Quantum to modernize Firefox's performance
- NSS (Network Security Services) -- the cryptographic library Firefox uses for TLS connections, also used by other Mozilla products
- GTK -- the GUI toolkit used on Linux for rendering the browser chrome and interface elements
- Cairo and Pango -- graphics and text rendering libraries used on Linux for drawing content correctly
- Rust -- a significant portion of Firefox's internals have been rewritten in Rust as part of Project Quantum, and the Rust compiler is itself entirely open source

What Firefox Has Enabled and Inspired

Firefox's most significant contribution to the broader ecosystem may not be the browser itself but what it forced other browsers to do. When Firefox showed that users would switch for a faster, more standards-compliant browser, every other vendor had to respond. Internet Explorer's decline after Firefox's 2004 launch is one of the clearest examples of open-source competition improving a market that had stagnated.

The Gecko rendering engine, which powers Firefox, is the basis for several other tools. Tor Browser is built directly on Firefox and inherits its privacy and security architecture. SeaMonkey, the continuation of the original Mozilla Suite, is also built on Gecko.

Mozilla also created Rust, the programming language. Rust started as an internal research project to write safer browser code with fewer memory safety bugs. It has since grown into one of the most adopted systems programming languages in the world, used by Linux kernel developers, Microsoft, Google, and Amazon. That is a contribution to open source that extends far beyond the browser itself.

Community and Governance

Mozilla's governance structure is unusual. The Mozilla Foundation is a nonprofit that owns Mozilla Corporation, a for-profit subsidiary. The Foundation sets the mission and values. The Corporation handles product development and generates revenue, primarily through search engine deals. Google pays Mozilla several hundred million dollars annually to be the default search engine in Firefox. That deal is both Mozilla's financial lifeline and its biggest governance tension.

The developer community communicates through Mozilla's Bugzilla instance for bug tracking, Phabricator for code review, and Matrix for real-time chat. Mozilla also maintains MDN Web Docs, which has become one of the primary references for web development standards regardless of which browser developers are targeting.

Relation to the LAMP Stack

Firefox is the client side of the web, not a component of the LAMP stack itself. LAMP describes the server side: Linux, Apache, MySQL, and PHP or Python or Perl. Firefox is the browser that connects to those servers and renders what they return.

That said, Firefox is deeply connected to the web standards that LAMP-based applications depend on. Its strict implementation of HTML, CSS, and JavaScript standards pushed web developers to write cleaner, more portable code rather than relying on IE-specific behavior. Its developer tools, which include an inspector, debugger, network monitor, and performance profiler, are part of the daily toolchain for many developers building applications on LAMP servers.

Part D -- Open Source vs Proprietary (Critical Analysis)

Comparison Table: Firefox vs Google Chrome

Dimension	**Firefox (Open Source / MPL 2.0)**	**Google Chrome (Proprietary)**
Cost	Free to download and use. No licensing fees.	Free to download. Revenue comes indirectly from Google's advertising ecosystem.
Security	Full source code is public. Any researcher can audit the complete codebase and verify fixes.	Chromium base is open but Chrome includes proprietary layers. You cannot fully audit the binary you are running.
Support	Community forums, Mozilla support site, MDN docs. No paid tier from Mozilla directly.	Google documentation and community support. Enterprise support via Google Workspace agreements.
Freedom to modify	Full MPL 2.0 rights to fork, modify, and redistribute. Tor Browser exercises this directly.	You can fork Chromium. The proprietary parts of Chrome cannot be modified or redistributed.
Community vs corporate control	Mozilla Foundation is a nonprofit. Decisions are supposed to serve the open web mission rather than shareholder returns.	Entirely controlled by Google, whose primary business is advertising. Product decisions reflect that relationship.
Overall verdict	Better for privacy, transparency, and the open web. Slightly behind on performance benchmarks and some compatibility.	Faster, dominant market share, excellent Google integration. But you are trusting a company that profits from tracking behavior.

Verdict

Given everything covered in this audit, I would recommend Firefox for any deployment where user privacy and code transparency are priorities. That covers most personal use, most small business use, and any context where the people responsible for the system want to be able to verify what the browser is actually doing. The fact that you can read the source code is not just a philosophical point. It is a practical one. When a security researcher finds something wrong, the fix is visible and you can verify it was actually applied rather than taking a company's word for it.

The honest counterpoint is that Firefox has been losing market share for years. Chrome holds over 60 percent of the global browser market and Firefox sits at roughly 3 to 4 percent. Part of that is Google's distribution advantages, including being the default on Android and on Chrome OS. Part of it is that Chrome is genuinely fast and well-integrated with services many people already use. If someone needs maximum compatibility for internal tools built specifically for Chrome, Firefox sometimes creates friction.

Would I contribute to it? Yes. MDN Web Docs is one of the most useful resources in web development and contribution there is accessible even for beginners. The source code is on GitHub and Mozilla maintains good documentation for new contributors. For a project that is genuinely trying to keep the web from being entirely controlled by a handful of advertising companies, that seems worth the time.

Shell Script Documentation (Units 2 & 5)

All five scripts are submitted as .sh files in the GitHub repository. Each is documented below with the full code, an explanation of what it does, and the shell scripting concepts it demonstrates.

Script 1: System Identity Report

Displays a welcome screen with the current Linux distribution name and kernel version, the logged-in user and their home directory, system uptime and current date and time, and a note about the open-source license covering both the OS and Firefox.

Shell concepts used

- Variables to store system information
- Command substitution with $() to capture command output into variables
- Reading /etc/os-release for portable distribution detection
- Formatted output using echo

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Devang Atiyolil | Course: Open Source Software
# Description: Welcome screen showing system info and OSS license details

STUDENT_NAME="Devang Atiyolil"
SOFTWARE_CHOICE="Firefox"
```

```
# Command substitution: run command and store output in variable
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
CURRENT_DATE=$(date '+%A, %d %B %Y %H:%M:%S')
HOME_DIR=$HOME

# Detect distro name from /etc/os-release (present on all modern distros)
if [ -f /etc/os-release ]; then
    DISTRO=$(grep ^PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '"')
else
    DISTRO="Unknown Linux Distribution"
fi

echo "========================================================="
echo "        Open Source Audit -- $STUDENT_NAME"
echo "        Chosen Software  : $SOFTWARE_CHOICE"
echo "========================================================="
echo "  Distribution : $DISTRO"
echo "  Kernel       : $KERNEL"
echo "  Logged in as : $USER_NAME"
echo "  Home Dir     : $HOME_DIR"
echo "  Uptime       : $UPTIME"
echo "  Date & Time  : $CURRENT_DATE"
echo ""
echo "  License Note : The Linux kernel is covered under"
echo "                 GNU General Public License v2 (GPL v2)."
echo "  Firefox      : Mozilla Public License 2.0 (MPL 2.0)."
echo "========================================================="
```

Script 2: FOSS Package Inspector

Checks whether Firefox is installed on the system, retrieves package details using the appropriate package manager (dpkg or rpm), and uses a case statement to print a philosophy note about the chosen software.

Shell concepts used

- if-then-else to branch based on package installation status
- command -v to detect which package manager is available
- case statement covering multiple package names
- Pipes with grep to filter relevant lines from package output

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Devang Atiyolil | Course: Open Source Software
# Description: Checks if Firefox is installed and prints package details and
#              a philosophy note about open-source software.

PACKAGE="firefox"

echo "========================================"
echo " FOSS Package Inspector"
echo "========================================"
echo ""
```

```
# Detect package manager and check installation status
if command -v dpkg &>/dev/null; then
    # Debian/Ubuntu based system
    if dpkg -l "$PACKAGE" 2>/dev/null | grep -q "^ii"; then
        echo "[INSTALLED] $PACKAGE found on this system."
        echo ""
        # Show version, homepage, description from dpkg
        dpkg -s "$PACKAGE" 2>/dev/null | grep -E "^(Version|Homepage|Description)" | sed
's/^/  /'
    else
        echo "[NOT FOUND] $PACKAGE is not installed."
        echo "  Install with: sudo apt install $PACKAGE"
    fi
elif command -v rpm &>/dev/null; then
    # RHEL/Fedora/CentOS based system
    if rpm -q "$PACKAGE" &>/dev/null; then
        echo "[INSTALLED] $PACKAGE found on this system."
        echo ""
        rpm -qi "$PACKAGE" | grep -E "Version|License|Summary" | sed 's/^/  /'
    else
        echo "[NOT FOUND] $PACKAGE is not installed."
        echo "  Install with: sudo dnf install $PACKAGE"
    fi
else
    # Fallback: check if binary exists in PATH
    if command -v firefox &>/dev/null; then
        echo "[FOUND] firefox binary at $(which firefox)"
    else
        echo "[NOT FOUND] Package manager not detected. Firefox not found."
    fi
fi

echo ""
echo "========================================"
echo " Philosophy Notes"
echo "========================================"
echo ""

# Case statement: print a note about the package's open-source significance
case $PACKAGE in
    firefox|firefox-esr)
        echo "  Firefox: A nonprofit's browser proving the open web is worth fighting for."
        echo "  It exists because someone decided user freedom matters more than market
share." ;;
    httpd|apache2)
        echo "  Apache: the web server that helped build the open internet."
        echo "  Collaborative and volunteer-built, it outperformed everything proprietary."
;;
    mysql|mariadb)
        echo "  MySQL/MariaDB: open source at the core of millions of applications."
        echo "  MariaDB is itself a story of a community fork defending its own freedom."
;;
    vlc)
        echo "  VLC: built by Paris students who just needed to stream video."
        echo "  Now it plays literally everything. A gift to the whole world." ;;
    git)
        echo "  Git: Linus built it when a proprietary tool revoked its free license."
        echo "  Best tools come from refusing to accept closed-source limits." ;;
```

```
    python3|python)
        echo "  Python: shaped entirely by its community through open proposals."
        echo "  Good design plus open collaboration eventually dominates every field." ;;
    *)
        echo "  $PACKAGE: open source, free to use, free to improve, free to share." ;;
esac

echo ""
echo "========================================"
```

Script 3: Disk and Permission Auditor

Loops through a list of important system directories and reports the permissions, owner, and disk usage for each using a for loop. Also checks whether Firefox's user config directory exists and explains the security implications of its permissions.

Shell concepts used

- for loop over a bash array of directory paths
- ls -ld combined with awk to extract permission string and owner fields
- du -sh combined with cut for human-readable size output
- if [-d] conditional to check directory existence before acting on it

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Devang Atiyolil | Course: Open Source Software
# Description: Audits permissions and disk usage for key system directories
#              and checks Firefox's user profile directory.

# Array of directories to audit
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "========================================"
echo " Directory Audit Report"
echo " Generated: $(date '+%d %B %Y %H:%M:%S')"
echo "========================================"
echo ""
printf "%-20s %-28s %-10s\n" "Directory" "Permissions (Owner)" "Size"
echo "----------------------------------------------------------------"

# Loop through each directory in the array
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        # awk extracts: field 1 = permissions, field 3 = user, field 4 = group
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3":"$4}')
        # du -sh gives human-readable size; suppress permission errors with 2>/dev/null
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        printf "%-20s %-28s %-10s\n" "$DIR" "$PERMS ($OWNER)" "${SIZE:-N/A}"
    else
        printf "%-20s %s\n" "$DIR" "[directory does not exist on this system]"
    fi
done

echo ""
echo "========================================"
```

```
echo " Firefox Config Directory Check"
echo "========================================="
echo ""

# Firefox stores user profiles here by default
FIREFOX_CONFIG="$HOME/.mozilla/firefox"

if [ -d "$FIREFOX_CONFIG" ]; then
    echo "  Found: $FIREFOX_CONFIG"
    echo ""
    ls -ld "$FIREFOX_CONFIG"
    echo ""
    echo "  This directory holds saved passwords, cookies, and session data."
    echo "  Permissions should be 700 (owner only). If others can read this,"
    echo "  it is a real privacy and security risk on shared systems."
else
    echo "  Firefox config not found at $FIREFOX_CONFIG"
    echo "  Firefox is either not installed or has never been launched by this user."
fi

echo ""
echo "========================================="
```

Script 4: Log File Analyzer

Accepts a log file path and optional keyword as command-line arguments. Reads the file line by line using a while-read loop, counts how many lines contain the keyword, and prints the last five matching lines. Includes a retry mechanism if the file is found to be empty.

Shell concepts used

- Command-line arguments via $1 and $2 with a default value fallback
- while IFS= read -r loop for safe, line-by-line file reading
- if-then inside the loop for per-line keyword matching using grep -iq
- Counter variables with arithmetic expansion $(())
- Bash array to accumulate matches and slice the last 5 with a C-style for loop

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: Devang Atiyolil | Course: Open Source Software
# Usage: ./script4_log_analyzer.sh /path/to/logfile [keyword]
# Description: Counts keyword occurrences in a log file and shows last 5 matches.

LOGFILE=$1
KEYWORD=${2:-"error"}   # Default to "error" if no keyword argument given
COUNT=0
MATCHES=()              # Array to store all matching lines

# Validate: file path must be provided
if [ -z "$LOGFILE" ]; then
    echo "Usage: $0 <logfile> [keyword]"
    echo "Example: $0 /var/log/syslog error"
    exit 1
fi

# Validate: file must exist
```

```
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File '$LOGFILE' not found."
    exit 1
fi

# Do-while style retry if file is empty (-s checks non-zero size)
RETRY=0
while [ ! -s "$LOGFILE" ]; do
    if [ $RETRY -ge 2 ]; then
        echo "File '$LOGFILE' is empty after $RETRY retries. Exiting."
        exit 1
    fi
    echo "Warning: file appears empty. Retry $((RETRY+1))/2..."
    sleep 1
    RETRY=$((RETRY + 1))
done

echo "========================================"
echo " Log File Analyzer"
echo " File   : $LOGFILE"
echo " Keyword: $KEYWORD"
echo "========================================"
echo ""

# Read file line by line using while-read loop
# IFS= prevents leading/trailing whitespace stripping
# -r prevents backslash interpretation
while IFS= read -r LINE; do
    # Case-insensitive (-i) keyword search, suppress output with -q
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
        MATCHES+=("$LINE")
    fi
done < "$LOGFILE"

echo "  Total lines scanned  : $(wc -l < "$LOGFILE")"
echo "  Keyword matches found: $COUNT"
echo ""

if [ $COUNT -gt 0 ]; then
    echo "  Last 5 matching lines:"
    echo "  ----------------------------------------"
    TOTAL=${#MATCHES[@]}
    # Start index: either 5 from the end, or 0 if fewer than 5 matches
    START=$(( TOTAL > 5 ? TOTAL - 5 : 0 ))
    for (( i=START; i<TOTAL; i++ )); do
        echo "  >> ${MATCHES[$i]}"
    done
else
    echo "  No lines matched the keyword '$KEYWORD'."
fi

echo ""
echo "========================================"
```

Script 5: Open Source Manifesto Generator

Interactively asks the user three questions, composes a personalised open source philosophy statement from their answers, saves it to a .txt file named after the current user, and displays it on screen. The note on aliases demonstrates understanding of where aliases work and why functions are preferred in scripts.

Shell concepts used

- read -p for interactive user input with inline prompts
- String concatenation across multiple echo statements to compose a paragraph
- Writing to a file using > to create and >> to append
- date command with format string for timestamps
- cat to display the completed output file

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Devang Atiyolil | Course: Open Source Software
# Description: Generates a personal open source manifesto from user input
#              and saves it to a .txt file.
#
# Note on aliases: in interactive shells you can define shortcuts like:
#   alias today='date "+%d %B %Y"'
# But aliases do not expand in non-interactive scripts unless you run
# shopt -s expand_aliases first. For portability, functions or direct
# commands are used here instead.

echo "========================================"
echo " Open Source Manifesto Generator"
echo " For: $(whoami)"
echo "========================================"
echo ""
echo "Answer three questions to generate your manifesto."
echo ""

# Collect user input with read -p (inline prompt)
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

# Capture current date for the header
DATE=$(date '+%d %B %Y')

# Output filename based on current username
OUTPUT="manifesto_$(whoami).txt"

# Write the manifesto to file
# > creates or overwrites the file; >> appends each subsequent line
echo "========================================" > "$OUTPUT"
echo " OPEN SOURCE MANIFESTO"                  >> "$OUTPUT"
echo " Written by: $(whoami)"                  >> "$OUTPUT"
echo " Date: $DATE"                            >> "$OUTPUT"
echo "========================================"  >> "$OUTPUT"
echo ""                                          >> "$OUTPUT"

# Main paragraph built from the three user answers
echo "I believe in software that is built to be shared."   >> "$OUTPUT"
echo ""                                                     >> "$OUTPUT"
```

```
echo "Every day I rely on $TOOL, a tool that someone"      >> "$OUTPUT"
echo "built and gave away so that others could build"      >> "$OUTPUT"
echo "further without starting from zero. That act of"     >> "$OUTPUT"
echo "choosing to share rather than hoard is what"         >> "$OUTPUT"
echo "$FREEDOM means to me in the context of software."    >> "$OUTPUT"
echo ""                                                     >> "$OUTPUT"
echo "I want to build $BUILD one day and release it"       >> "$OUTPUT"
echo "as open source, so that someone I will never meet"   >> "$OUTPUT"
echo "can use it, improve it, and pass it on."             >> "$OUTPUT"
echo ""                                                     >> "$OUTPUT"
echo "Open source is not just a license. It is a choice"   >> "$OUTPUT"
echo "about what kind of world we want to live in."        >> "$OUTPUT"
echo "I choose open."                                      >> "$OUTPUT"
echo ""                                                     >> "$OUTPUT"
echo "-- $(whoami), $DATE"                                 >> "$OUTPUT"
echo "========================================"            >> "$OUTPUT"

echo ""
echo "Manifesto saved to: $OUTPUT"
echo ""
echo "-------- Your Manifesto --------"
cat "$OUTPUT"
echo "--------------------------------"
```

Conclusion

This audit started with a browser and ended with a question about what kind of infrastructure the internet should run on. Firefox is interesting not because it is the best browser by every metric right now, but because it is the main browser whose existence is justified by a mission rather than a business model.

The license analysis showed that MPL 2.0 is a thoughtful middle ground, more protective than MIT but more practical for adoption than GPL. The Linux footprint section made it clear how cleanly Firefox fits into the Linux filesystem model, and how the separation between system files and user data is a concrete security benefit, not just a convention. The ecosystem section revealed that Mozilla's contributions extend well beyond the browser into Rust, web standards documentation, and the general health of the open web.

The comparison with Chrome was the most honest part of writing this. Chrome is faster and more compatible in some situations. But you are accepting a trade when you use it: you are trusting a company whose entire business depends on understanding your behavior online. Firefox's source being open does not automatically make it trustworthy, but it does make trust verifiable rather than assumed. That is a meaningful difference.

The five shell scripts reinforced something this course has been building toward: that open-source philosophy and practical Linux skills are not separate topics. Every command used in those scripts, every library it calls, every shell interpreting the syntax, is itself built on decades of people choosing to share their work. Understanding that is not a footnote to a technical education. It is the foundation everything else sits on.

References

- Mozilla Foundation. Mozilla Public License 2.0. mozilla.org/MPL/2.0/
- GNU Project. The Free Software Definition. gnu.org/philosophy/free-sw.html
- Open Source Initiative. The Open Source Definition. opensource.org/osd
- Eric S. Raymond. The Cathedral and the Bazaar. catb.org/~esr/writings/cathedral-bazaar
- Mozilla. Firefox Source Code. searchfox.org
- Mozilla Developer Network. MDN Web Docs. developer.mozilla.org
- William Shotts. The Linux Command Line. linuxcommand.org
- SPDX License List. spdx.org/licenses